EECS 348 Assignment 4 UML Use Case Analysis:

Use a GenAI of your choice to help you create a UML Use Case Diagram for this assignment with the level of detail given in the first UML Use Case Modeling lecture:

A)

The use case I identified is “Solve Sudoku Puzzles.” Its main goal is to allow the User to solve five Sudoku puzzles and view all valid solutions. I chose this use case because it represents a complete goal that provides value to the User, unlike smaller actions such as reading a file or checking a cell. The system is the Sudoku solver program (Assignment4.cpp, compiled into an executable). The primary actor is the User, who runs the program, while the puzzle files (puzzle1.txt through puzzle5.txt) provide the input. The use case involves three required tasks: loading the puzzle, checking the clues, and printing the results. An alternative outcome occurs when the system cannot find a solution, in which case it reports that no solution exists.

B)

This use case describes how the User runs the program to solve five Sudoku puzzles. The system reads each file and uses recursive backtracking to fill empty cells with valid digits. If a choice leads to a dead end, the system removes that digit and tries another possibility. For each puzzle, the program displays the original grid, all valid solutions, and the total number of solutions found. If no solution exists, it displays “No solution found.” The program processes all five files before terminating.

C)

1. The User runs the Sudoku solver executable from the folder containing the five puzzle files.
2. The system opens the first puzzle file and reads its nine rows of nine symbols.
3. The system checks that the file contains valid symbols: digits 1–9 or underscores representing empty cells.
4. The system displays the original puzzle.
5. The system searches for valid solutions using recursive backtracking, checking each candidate digit against the corresponding row, column, and 3×3 block.
6. When a candidate leads to a dead end, the system removes it and tries another candidate.
7. Whenever a complete solution is found, the system prints the solution and continues searching for additional solutions.
8. The system displays the total number of solutions found or prints “No solution found” if no solutions exist.
9. The system repeats these steps for the remaining puzzle files and terminates after processing all five.

Alternate flows:

- A1: If a file is missing or contains invalid data, the system displays an error message and proceeds to the next file.
- A2: If the puzzle contains conflicting clues, the system reports that no solution exists and skips the search.
- A3: If the search finishes without finding any solutions, the system prints “No solution found.”

D)

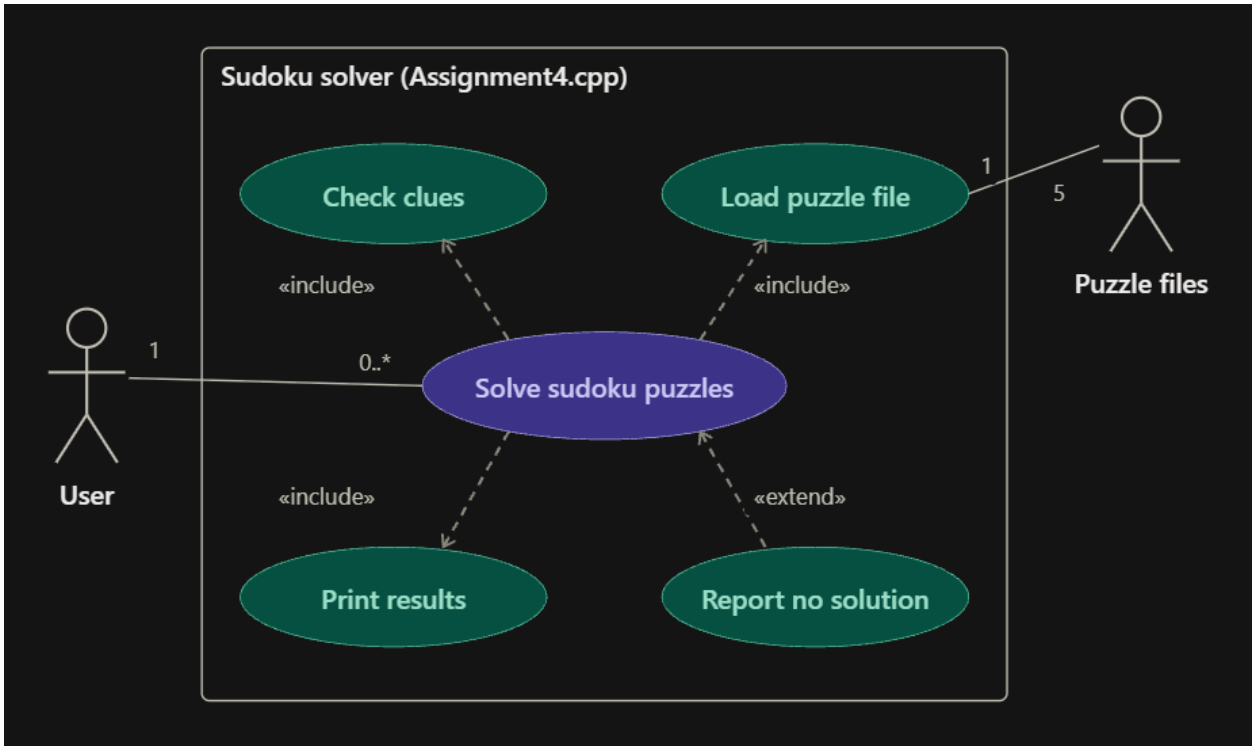
The User starts the program in a directory containing the five Sudoku puzzle files. No additional input is required. The system processes each file in order, displays its name, and reads the puzzle as a 9×9 grid. If a file cannot be opened, contains fewer than 81 symbols, or includes invalid symbols, the system displays an error message and continues to the next file.

After successfully reading a puzzle, the system displays the original grid. It then checks whether any given digits violate the Sudoku rules by repeating within a row, column, or 3×3 block. If conflicting clues are detected, the system reports that no solution exists and moves on to the next puzzle.

For a valid puzzle, the system begins a recursive depth-first backtracking search. It locates an empty cell and tries each digit from 1 to 9, placing only digits that do not violate the Sudoku rules. The system recursively searches the remaining cells. If a choice prevents the puzzle from being completed, the system resets that cell and tries another candidate. Whenever all cells are filled successfully, the system prints the completed grid and increments the solution count. The search continues until every possible solution has been explored.

After the search finishes, the system displays the total number of solutions found or prints “No solution found” if the count is zero. It then repeats the process for the remaining files. Once all five puzzles have been processed, the program terminates. For the supplied puzzles, puzzles 1–3 each have one solution, puzzle 4 has no solution, and puzzle 5 has two solutions.

GenAI Used: Claude (Claude.ai)


Sudoku solver (Assignment4.cpp)
Check clues
Load puzzle file
Solve sudoku puzzles
Print results
Report no solution
«include»
«include»
«include»
«extend»
1
0..*
1
5
User
Puzzle files